MeetFlow

Browser-to-Browser Video Conferencing Application

High-Level Design (HLD)

MeetFlow Engineering Team

June 21, 2026

Contents

1 Introduction

1.1 Project Objective

The objective of the **MeetFlow** project is to design and develop a secure, robust, and highly responsive browser-based real-time communication platform. MeetFlow allows users to create and join meetings, communicating seamlessly via high-definition video, audio, text chat, screen sharing, and peer-to-peer file transfer. Crucially, the platform operates purely inside modern web browsers without requiring external plugins or native desktop installations.

1.2 Scope of the Document

This High-Level Design (HLD) document outlines the architectural blueprint, data models, signaling protocols, and component relationships for MeetFlow. It details how WebRTC, Socket.IO, Node.js, and MySQL combine to support a secure, responsive application.

1.3 Release Phasing

To manage complexity and iterate effectively, the application is divided into three major milestones:

- **Phase 1 (One-to-One Baseline):** Direct browser-to-browser audio/video streams, meeting room creation and joining, basic signaling structure.
- **Phase 2 (Collaboration Features):** Real-time text chat, file transfers (via WebRTC Data Channels), screen sharing, and participant roster controls.
- **Phase 3 (Production Readiness):** Database persistence, user account registration/login, JWT authentication, STUN/TURN (Coturn) integration, multi-party SFU topology research, and deployment configurations.

2 System Architecture

MeetFlow relies on a hybrid architectural topology. While signaling, user administration, and metadata transactions use a centralized Client-Server model, media streams (audio, video, files) are transmitted directly between browsers using WebRTC's peer-to-peer (P2P) connections.

2.1 Architectural Components

The system comprises the following primary building blocks:

1. **Frontend Application (React & Vite):** A responsive single-page application (SPA) rendering UI panels, requesting camera/microphone access via the browser WebRTC API, and handling peer connection negotiations.
2. **Backend Web Server (Node.js & Express):** Exposes RESTful APIs for user management, authentication, and meeting space configurations.
3. **Signaling Server (Socket.IO):** A stateful WebSocket-based messaging node facilitating the initial exchange of session descriptors (SDP) and Network ICE candidates.
4. **Database (MySQL):** Stores relational credentials, profiles, meeting logs, and structural data.
5. **STUN/TURN Servers (Coturn):** Provides NAT traversal capabilities. STUN identifies public IPs, while TURN acts as a media relay for users behind strict symmetric firewalls.

2.2 System Architecture Diagram

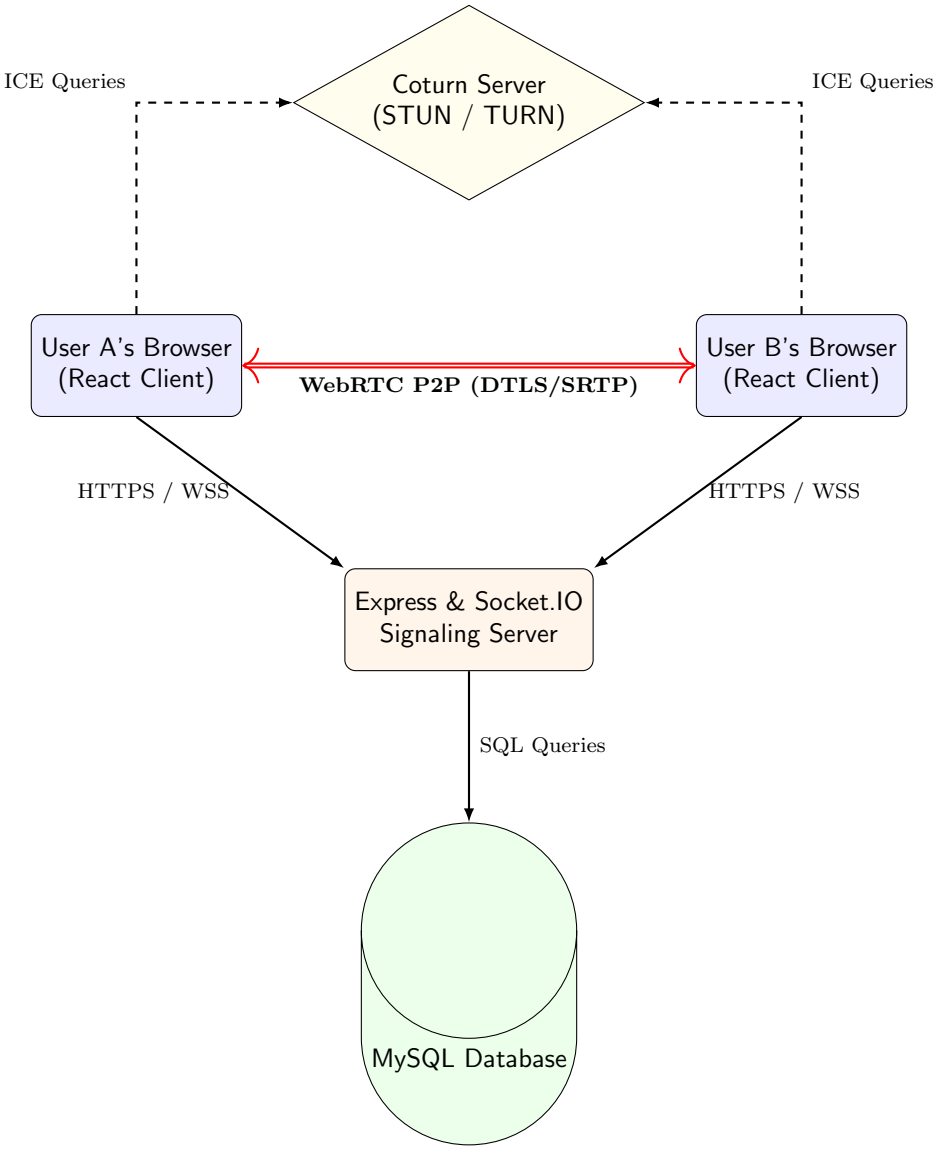

Figure 1: MeetFlow High-Level Architecture Overview

3 WebRTC Signaling & Connection Flow

Before User A and User B can streams media directly, they must undergo the signaling phase. The signaling server acts as a post-office, forwarding SDP offers, SDP answers, and ICE Candidates.

3.1 Signaling Sequence

The exact steps required to instantiate a peer-to-peer session are detailed below:

1. **Room Entry:** User A joins a meeting room by issuing a `join-room` event.
2. **Discovery:** When User B enters, the signaling server triggers a user-connected notification to User A.
3. **Offer Creation:** User A creates a Local Session Description Protocol (SDP) offer containing codec information, media attributes, and cryptographic parameters, and sends it to the Signaling Server.
4. **Offer Relay:** The Signaling Server routes the SDP offer directly to User B.
5. **Answer Creation:** User B registers User A's offer as its remote description, requests its local audio/video streams, generates an SDP answer, and routes it back through the Signaling Server.
6. **Answer Relay:** The Signaling Server forwards the SDP answer to User A.
7. **ICE Exchange:** As User A and B discover network pathways (via STUN/TURN queries), they stream their network configurations (`ice-candidate` events) through the Signaling Server to each other.
8. **P2P Connection:** Once descriptions are applied and at least one ICE path is validated, the WebRTC peer connection shifts to the connected state, and direct media streaming commences.

3.2 Signaling Sequence Diagram

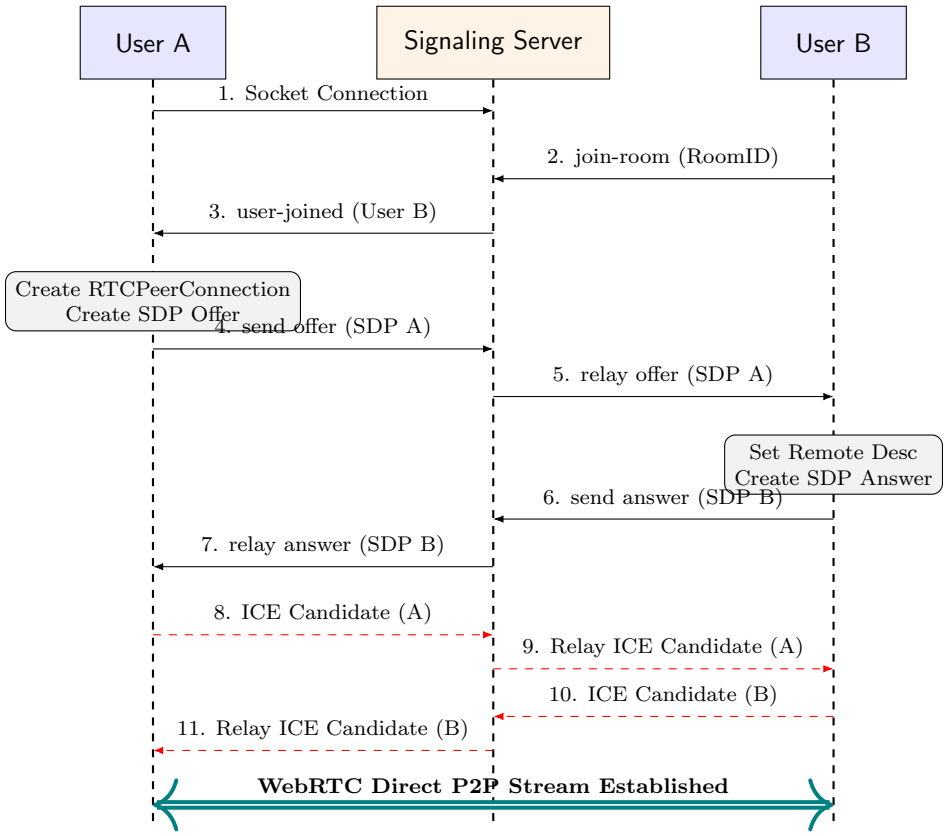

Figure 2: Signaling Protocol Sequence Flow

4 Data Model & Database Design

MeetFlow relies on a relational database system (MySQL) to maintain user states, profiles, audit records, and history logging.

4.1 Entity-Relationship Model (Conceptual)

The core Entities are:

- **User**: Holds basic identification, security hashes, and settings.
- **Meeting**: Tracks unique physical session identifier, creator metadata, active status, and timestamp limits.
- **ParticipantSession**: Intermediary join table auditing which users joined what meeting and at what times.
- **ChatMessage**: Logs text logs sent inside room contexts.

4.2 Database Schema Definitions

4.2.1 Users Table

Tracks registration details and authorization criteria.

4.2.2 Meetings Table

Tracks unique session tokens and runtime status.

Column Name	Data Type	Attributes	Description
id	INT	PK, AUTO_INCREMENT	Unique identifier for the us
username	VARCHAR(50)	UNIQUE, NOT NULL	User-selected screen handle
email	VARCHAR(100)	UNIQUE, NOT NULL	Verified user email address.
password_hash	VARCHAR(255)	NOT NULL	Bcrypt password digest.
avatar_url	VARCHAR(255)	NULL	URI locating user's custom
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date of account generation.
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Date of last profile alteratio

Table 1: Users Table Schema

Column Name	Data Type	Attributes	Description
id	VARCHAR(12)	PK	Unique URL identifier (e.g., ABC
host_id	INT	FK (users.id)	Identifies the user who initiated
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Start timestamp of the session.
ended_at	TIMESTAMP	NULL	Termination timestamp of the s
status	ENUM	('active', 'ended')	Current execution state of the r

Table 2: Meetings Table Schema

4.2.3 Meeting_Participants Table

Audits individual attendees for analytics and history display.

Column Name	Data Type	Attributes	Description
id	INT	PK, AUTO_INCREMENT	Unique row ID.
meeting_id	VARCHAR(12)	FK (meetings.id), NOT NULL	Target meeting room.
user_id	INT	FK (users.id), NOT NULL	Participating member.
joined_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Entrance timestamp.
left_at	TIMESTAMP	NULL	Leaving timestamp.

Table 3: Meeting Participants Table Schema

5 API & WebSocket Interface Specifications

5.1 RESTful Web APIs

5.1.1 User Registration

- **Endpoint:** `POST /api/auth/register`
- **Description:** Creates a new user record.
- **Request Payload:**

```
{
  "username": "johndoe",
  "email": "johndoe@example.com",
  "password": "SecurePassword123!"
}
```

- **Response Payload (Success 201 Created):**

```
{
  "message": "User registered successfully",
  "userId": 101
}
```

5.1.2 User Login

- **Endpoint:** `POST /api/auth/login`
- **Description:** Validates credentials and returns a JWT.
- **Request Payload:**

```
{
  "email": "johndoe@example.com",
  "password": "SecurePassword123!"
}
```

- **Response Payload (Success 200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 101,
    "username": "johndoe",
    "email": "johndoe@example.com",
    "avatarUrl": null
  }
}
```

5.1.3 Create Meeting

- **Endpoint:** `POST /api/meeting/create`
- **Description:** Instantiates an active meeting record. Requires authorization header: `Authorization: Bearer <token>`.
- **Response Payload (Success 200 OK):**

```
{
  "meetingId": "ABCD-EFGH",
  "hostId": 101,
  "createdAt": "2026-06-21T22:58:55.000Z"
}
```

5.2 WebSocket Signaling Events

These events are transmitted using Socket.IO during an active video conference.

- `join-room` **(Client → Server):** Initiated when the client navigates into a room.

```
{
  "roomId": "ABCD-EFGH",
  "userId": 101,
  "username": "johndoe"
}
```

- `offer` **(Client → Server → Target Client):** Transmits the local SDP offer.

```
{
  "targetSocketId": "socket_id_of_recipient",
  "offer": {
    "type": "offer",
    "sdp": "v=0\r\no=alice 2890844526..."
  }
}
```

- `answer` **(Client → Server → Target Client):** Relays the SDP answer.

```
{
  "targetSocketId": "socket_id_of_recipient",
  "answer": {
    "type": "answer",
    "sdp": "v=0\r\no=bob 2890844527..."
  }
}
```

- `ice-candidate` **(Client → Server → Target Client):** Streams discovered ICE pathways.

```
{
  "targetSocketId": "socket_id_of_recipient",
  "candidate": {
    "candidate": "candidate:842163049 1 udp 1686052607 192.168.1.2
       55301 typ host...",
    "sdpMid": "0",
    "sdpMLineIndex": 0
  }
}
```

6 Security, Performance & Reliability

6.1 Security Engineering

Security is a foundational tenet of MeetFlow's real-time communication stack.

- **Transport Security:** All traffic (HTTPS, WSS) is encrypted using Transport Layer Security (TLS 1.3) to prevent eavesdropping and hijacking.
- **Media Encryption:** WebRTC enforces mandatory end-to-end media encryption. Signaling channels exchange temporary cryptographic keys, and the media streams are secured using **DTLS** (Datagram Transport Layer Security) and **SRTP** (Secure Real-time Transport Protocol).
- **JWT Authentication:** RESTful API commands targeting meeting configurations require validating cryptographically signed JSON Web Tokens.
- **Credential Protection:** Password assets are hashed using the CPU-bound **Bcrypt** framework utilizing an index work factor of 12.

6.2 Performance Goals

- **Session Setup Latency:** Mean duration to transition from room ID entry to video visibility must remain lower than 5 seconds under normal network conditions.
- **Real-Time Network Latency:** One-way end-to-end media latency must stay below 200 ms. Jitter buffers adjust dynamically to smooth out stream variance.
- **Resolution & Bandwidth Adaptation:** Client implementations utilize `RTCRtpSender.setParameters(` or WebRTC's default simulcast capability to lower output resolutions dynamically based on local congestion detections.

6.3 Reliability and Fault Tolerance

- **Automatic Reconnection:** If the signaling WebSocket connection drops, Socket.IO is configured to execute exponential back-off reconnections.
- **ICE Restart:** If network changes occur (e.g., switching from Wi-Fi to cellular data), the client initiates a WebRTC ICE restart procedure, triggering a fresh offer-answer exchange over the active signaling channel.

7 User Interface & Meeting Layout

7.1 UI Pages Map

The web app is structured into pages detailed in the hierarchy below:

- **Landing Home Page:** Explains features with direct buttons to register or log in.
- **Dashboard:** Displays upcoming meetings, meeting history, profiles access, and a simple box to "Start Meeting" or "Join Meeting".
- **Meeting Room Screen:** The unified workspace where real-time video grid and utility sidebars render.

7.2 Meeting Room Design Grid Layout

The structural configuration of the meeting page is diagrammed below:

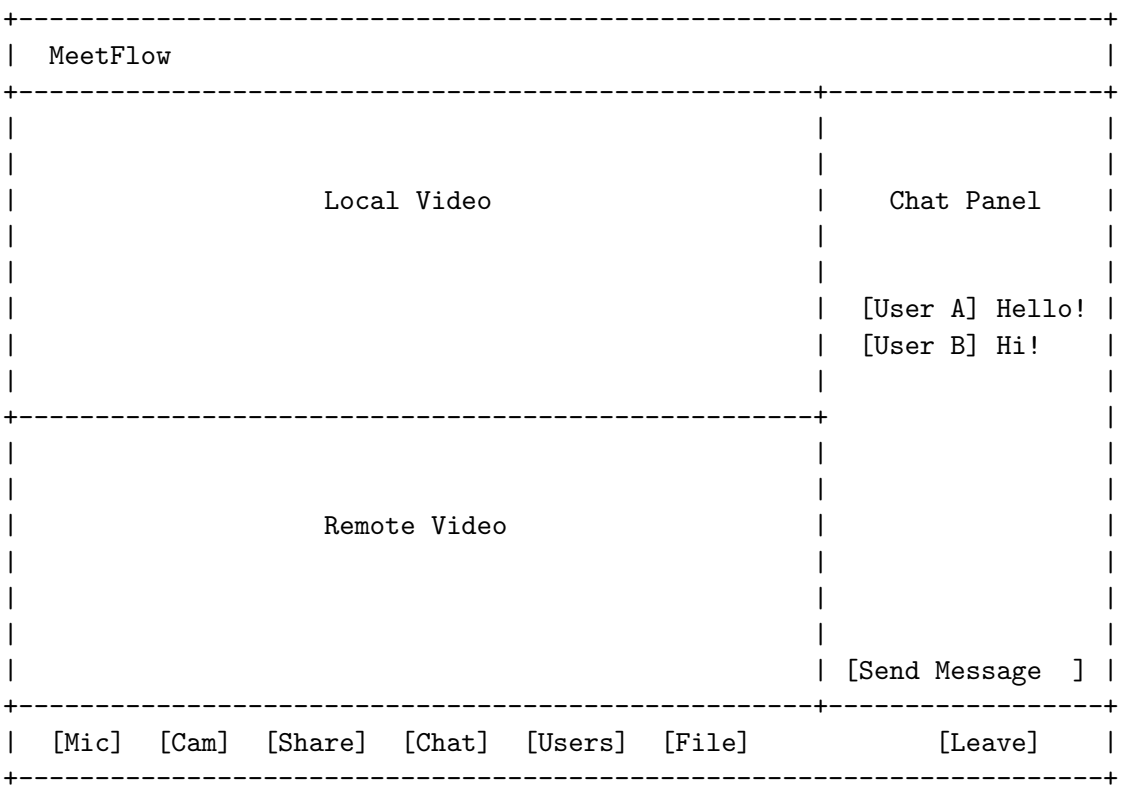

Figure 3: Visual Mockup Grid of the Meeting Room Screen

The layout comprises three distinct flex zones:

1. **Media Stage (Left Area):** Displays the feed of the local user and the incoming feed of the remote participant in a grid.
2. **Sidebar (Right Area):** Toggles dynamically between the Chat log and the active Participants list.
3. **Control Deck (Bottom Bar):** A floating, glassmorphic HUD bar exposing mute/unmute buttons, camera toggles, screen sharing buttons, file uploads, chat toggle, and a leave meeting button.